

SIBDAS: Desenvolvimento de um Sistema Web de Apoio ao Inventário Hospitalar de Equipamentos Médicos

21/06/2026

Docentes:
Pedro Manuel Sousa Guimarães



Realizado por:
Amadeu Campos 1240896

Engenharia Biomédica

Índice

- 1. Introdução ----- 4
- 2. Funcionalidades Desenvolvidas ----- 5
 - 2.1. Login através da Base de Dados ----- 5
 - 2.2. Logout e Validação da Sessão para Proteção do Backend ---- 5
 - 2.3. Dashboard ----- 6
 - 2.4. Inserção de Equipamentos, Componentes, Garantias, Localizações, Fornecedores e Documentos ----- 6
 - 2.5. Associação e Desassociação de Fornecedores ----- 7
 - 2.6. Listagem de Dados ----- 8
 - 2.7. Validação para Funções de Administrador ----- 8
 - 2.8. Eliminação e Recuperação de Dados ----- 9
 - 2.9. Edição de Equipamentos, Componentes, Garantias, Localizações, Fornecedores e Documentos ----- 9
 - 2.10. Edição de Texto do Frontend através do Backend ----- 10
 - 2.11. Pesquisa Avançada ----- 11
 - 2.12. Vista Detalhada de Equipamentos ----- 11
- 3. Base de Dados ----- 12
 - 3.1. Modelo Relacional ----- 12
 - 3.2. Restrições de Integridade ----- 13
 - 3.3. Exemplos de Consultas SQL ----- 14
- 4. Conclusão ----- 15

1. Introdução

No panorama atual das instituições de saúde, a gestão eficaz do parque tecnológico assume um papel crítico na qualidade da prestação de cuidados médicos. Contudo, em muitas unidades de média dimensão, a gestão do inventário de equipamentos clínicos ainda é efetuada de forma descentralizada e obsoleta, recorrendo a folhas de cálculo dispersas, registos manuais em papel e arquivos físicos não integrados.

Esta realidade é visível no caso em estudo: uma unidade hospitalar com um volume considerável de equipamentos médicos enfrenta graves lacunas operacionais. A ausência de uma plataforma centralizada traduz-se na duplicação e inconsistência de dados, na dificuldade em localizar dispositivos críticos em tempo útil, na falta de um histórico estruturado e na fragilidade documental face a auditorias ou processos de certificação de qualidade. Segundo as diretrizes da Organização Mundial da Saúde (OMS), um inventário devidamente estruturado e atualizado é o alicerce fundamental para o planeamento, controlo do ciclo de vida tecnológico e apoio à manutenção dos dispositivos médicos.

2. Funcionalidades Desenvolvidas

2.1. Login através da base de dados

No login.html, na parte pública, é pedido o nome do utilizador e a palavra-passe do mesmo. Essa informação é levada para a parte privada pelo método post para o login.php. Este script PHP processa a autenticação de utilizadores ligando-se a uma base de dados MySQL via PDO e gerindo o acesso ao painel de controlo de forma segura. O fluxo inicia-se com a configuração estrita da sessão através de diretivas que mitigam os riscos de ataques Cross-Site Scripting (XSS), bloqueando o acesso ao cookie via JavaScript através do parâmetro cookie_httponly e forçando a gestão da identidade exclusivamente por cookies com o use_only_cookies. Caso detete que o utilizador já possui uma sessão ativa, o código redireciona-o automaticamente para o painel principal, poupando-o de preencher o formulário novamente.

A comunicação com a base de dados é blindada contra ataques de SQL Injection ao desativar a emulação de consultas pelo PDO, o que obriga o sistema a utilizar Prepared Statements nativos que tratam qualquer texto introduzido estritamente como um dado. Além disso, a ligação é protegida por um bloco try/catch que intercepta falhas técnicas e as oculta do utilizador, exibindo apenas uma mensagem de erro genérica para não expor detalhes sensíveis da infraestrutura do servidor.

Quando os dados do formulário são submetidos via POST, o script limpa o nome de utilizador com a função trim e realiza uma consulta estruturada com um LEFT JOIN à tabela de administradores, permitindo identificar de imediato se a conta possui privilégios elevados. A validação da identidade é feita de forma criptográfica através da função password_verify, que compara o texto digitado com o hash seguro armazenado na base de dados, garantindo que as palavras-passe nunca sejam expostas em texto limpo.

Se as credenciais estiverem corretas, o script executa o comando session_regenerate_id(true) para alterar o identificador da sessão e neutralizar ataques de Fixação de Sessão (Session Fixation). Em seguida, armazena na superglobal \$_SESSION o ID do utilizador, o seu nome, o marcador de login e o seu nível de acesso (administrador ou utilizador comum), além de registar o carimbo de data/hora atual com time() para permitir futuros controlos de tempo limite por inatividade (timeout), concluindo com o redirecionamento para a página restrita. Se o login falhar, o utilizador é devolvido ao formulário com uma mensagem propositadamente genérica que indica que o utilizador ou a palavra-passe estão incorretos, aplicando o princípio da segurança por obscuridade para impedir que atacantes descubram nomes de utilizador válidos através de tentativas de força bruta.

2.2. Logout e Validação da sessão para proteção do backend

O logout.php implementa a funcionalidade de logout (terminar sessão), sendo responsável por encerrar de forma segura o acesso do utilizador ao sistema. Ele inicia o contexto da sessão atual, remove todas as variáveis guardadas na memória do servidor através da função session_unset e destrói completamente o registo da sessão com o session_destroy. Por fim, redireciona o utilizador para a página inicial do site passando um parâmetro na URL que sinaliza que a desconexão foi efetuada com sucesso, interrompendo imediatamente qualquer execução posterior com o comando exit.

Para além disso, no início de cada página acessível do backend foi implementado um bloco de código php que funciona como um filtro de controlo de acesso (segurança), concebido para proteger páginas restritas contra utilizadores não autenticados. O script ativa a sessão e verifica se a variável que confirma o login (\$_SESSION['logado']) existe e é verdadeira. Caso o utilizador não cumpra este requisito, o código assume que se trata de um acesso indevido e, por motivos de segurança, limpa e destrói preventivamente qualquer resíduo de sessão ativo antes de expulsar o utilizador para a página de login com um aviso de acesso restrito, garantindo através do exit que nenhuma informação confidencial da página protegida seja processada ou exibida.

```
session_start();

if (!isset($_SESSION['logado']) || $_SESSION['logado'] !== true) {

    // Por segurança, limpa qualquer resíduo de sessão que possa existir
    session_unset();
    session_destroy();

    // 3. Expulsar o intruso de volta para o formulário de login
    // Ajusta o caminho se o teu login.php estiver numa pasta acima (ex: ../login.php)
    header("Location: ../public/login.html?erro=restrito");
    exit; // Interrompe imediatamente a execução do resto da página
}
```

Figura 1 - Controlo de acesso.

```
<?php
session_start();

// Limpa todas as variáveis de sessão ativos
session_unset();

// Destrói a sessão no servidor
session_destroy();

// Redireciona de volta para o login com uma mensagem de sucesso
header("Location: ../public/index.php?status=desconectado");
exit; ?>
```

Figura 2 - Código para o término de sessão.

2.3. Dashboard

O script php estabelece uma ligação à base de dados através da extensão `mysqli_connect` e executa uma série de consultas SQL para extrair indicadores estatísticos em tempo real. Estas operações contabilizam o volume total de dispositivos de forma global e segmentada pelos seus estados operacionais, isolando as quantidades de equipamentos ativos, em manutenção ou inativos, além de computar o número total de fornecedores e de localizações físicas registadas no hospital.

Adicionalmente, o código realiza análises operacionais e de risco mais avançadas através de junções de tabelas. Ele calcula o total de garantias expiradas ao cruzar os dados dos equipamentos com os seus contratos através de um `INNER JOIN` filtrado pela data atual, e identifica falhas de conformidade ao contabilizar os equipamentos que não possuem qualquer documento associado usando um `LEFT JOIN` com a tabela de documentação. O script também agrupa o volume total de equipamentos por serviço ou departamento e isola especificamente a distribuição de dispositivos críticos pertencentes à categoria de "Suporte de vida" por cada setor da instituição.

Por fim, toda esta informação estruturada é injetada dinamicamente numa interface gráfica responsiva construída com o framework Bootstrap 5. O layout apresenta uma barra de navegação completa (que vai ser encontrada por praticamente todas as páginas do backend) com menus suspensos para a gestão avançada de equipamentos, componentes, localizações, fornecedores e documentos, seguida por cartões de estatísticas visualmente destacados com ícones da biblioteca Font Awesome para os principais indicadores numéricos. O Dashboard conclui-se com a renderização de duas tabelas dinâmicas que listam de forma ordenada a distribuição geral de equipamentos e o inventário de dispositivos de suporte de vida por departamento, oferecendo uma visão centralizada, intuitiva e imediata do estado real de todo o parque tecnológico hospitalar.

2.4. Inserir Equipamentos, Componentes, Garantias, Localizações, Fornecedores e Documentos

Em termos gerais, estes seis scripts de inserção de dados partilham uma mesma arquitetura de backend e o mesmo rigor no tratamento de segurança. Todos eles funcionam exclusivamente como controladores de submissão de formulários através do método POST, o que significa que qualquer tentativa de acesso direto pelo URL (método GET) resulta no bloqueio automático e no redirecionamento do utilizador para a página de origem. Para além disso, todos partilham as mesmas credenciais e configurações técnicas para estabelecer ligação ao servidor MySQL remoto e utilizam o sistema de sessões do PHP para armazenar mensagens flash de sucesso ou erro, que dão feedback visual ao utilizador após a operação. O ponto comum mais importante a nível de segurança é o uso sistemático de consultas preparadas (Prepared Statements), garantindo que os dados submetidos nos formulários sejam tratados estritamente como texto e nunca como comandos SQL executáveis, neutralizando assim falhas de SQL Injection. Há também uma rotina partilhada de sanitização básica, onde funções como `trim` limpam espaços em branco e operadores como `intval` ou `floatval` asseguram a integridade dos tipos numéricos.

Por outro lado, os ficheiros diferenciam-se significativamente pelas regras de negócio específicas de cada entidade hospitalar e pela complexidade das validações que executam individualmente. O script `inserir_localizacao.php` é o mais simples e direto de todos, limitando-se a validar se os quatro campos geográficos obrigatórios foram preenchidos antes de os gravar. Já o ficheiro `inserir_componente.php` destaca-se por ser o único focado numa relação de dependência hierárquica, associando um subcomponente a um equipamento pai que já deve existir no sistema. No caso do `inserir_fornecedor.php`, a sua particularidade reside na captura específica do código de erro 1062 do MySQL, que permite interceptar e tratar de forma amigável no frontend as tentativas de duplicação do NIF, uma vez que este campo é uma chave única na base de dados. O script `inserir_garantia_contrato.php` introduz lógica condicional de campos, em que parâmetros como a periodicidade e o tipo de contrato só passam a ser obrigatórios se o marcador de contrato de manutenção estiver ativo, validando também que a data de fim do contrato nunca seja anterior à sua data de início.

A complexidade aumenta no ficheiro `inserir_equipamento.php`, que apresenta a validação de datas mais robusta ao recorrer à classe `DateTime` do PHP para garantir que os formatos introduzidos são reais, bloqueando logicamente que o equipamento tenha uma data de aquisição no futuro ou que esta seja anterior ao seu próprio ano de fabrico. Por fim, o script `inserir_documento.php` afasta-se por completo da dinâmica dos restantes devido à necessidade de processar uploads de ficheiros físicos através da superglobal `$_FILES`. Este código assume a responsabilidade de verificar se a extensão do ficheiro é permitida (PDF, JPG, PNG), limitar o tamanho a um máximo de 5MB por segurança, gerar um nome único em hash MD5 baseado no tempo para evitar a sobreposição de ficheiros e criar o diretório de destino no servidor caso ele não exista. É também o único bloco que exige, de forma explícita e direta no topo do ficheiro, o controlo restrito de privilégios para verificar se o utilizador está logado no sistema antes de efetuar qualquer operação.

2.5. Associação e Desassociação de Fornecedores

Foi criado um script PHP focado no registo de dados numa tabela intermédia chamada `equipamento_fornecedor`, estabelecendo uma relação entre um equipamento e um fornecedor através de uma função contratual ou técnica específica. O fluxo inicia-se com a abertura da sessão e foca-se no processamento de requisições submetidas exclusivamente via método POST. Após conectar-se ao servidor MySQL, o código limpa e converte os IDs recebidos para números inteiros através de `intval` e remove espaços vazios do tipo de fornecedor. É aplicada uma dupla validação de segurança: primeiro verifica-se se os dados mínimos obrigatórios foram preenchidos e, de seguida, cruza-se o tipo de fornecedor submetido com um array de controlo que contém os quatro valores aceitáveis pelo campo ENUM, mitigando potenciais adulterações maliciosas no HTML do formulário. Caso passe os testes, a gravação é executada por intermédio de Prepared Statements, protegendo o sistema contra SQL Injection. O script conta ainda com um tratamento de erros dedicado para capturar o código 1062 do MySQL, alertando o utilizador através de uma mensagem flash se a associação já existir, concluindo com o encerramento das ligações e o redirecionamento para o painel de gestão.

O script `php_eliminar_associacao_fornecedor.php` expande este fluxo ao implementar um ecrã de confirmação visual e a posterior eliminação (desvinculação) da relação comercial entre o equipamento e o fornecedor. O script começa por reforçar a segurança do ecossistema ao validar se o utilizador possui uma sessão ativa e legítima com o marcador logado, limpando e destruindo preventivamente a sessão em caso de intrusão antes de o redirecionar para o login. Ao contrário do primeiro bloco, este script aceita requisições tanto por GET como por POST via `$_REQUEST`, extraíndo os parâmetros de identificação da associação. Quando a página é acedida inicialmente pelo método GET, o código executa uma consulta com junções (JOIN) para traduzir os IDs numéricos nos nomes reais do equipamento e da empresa fornecedora, injetando estes dados de forma legível numa caixa de aviso estilizada com Bootstrap 5 e ícones decorativos. Caso o utilizador avance e clique no botão de exclusão, o formulário submete os dados via POST com a variável `confirmar_eliminar`, disparando um comando DELETE via consulta preparada para apagar estritamente o vínculo na tabela intermédia, mantendo os registos originais das duas entidades completamente intactos e finalizando com o redirecionamento e feedback do sucesso da operação.

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['confirmar_eliminar'])) {  
    $sql = "DELETE FROM equipamento_fornecedor  
           WHERE equipamento_id = ? AND fornecedor_id = ? AND tipo_fornecedor = ?";  
    $stmt = mysqli_prepare($conn, $sql);  
    if ($stmt) {  
        mysqli_stmt_bind_param($stmt, "iis", $equipamento_id, $fornecedor_id, $tipo_fornecedor);  
        if (mysqli_stmt_execute($stmt)) {  
            mysqli_stmt_close($stmt);  
            mysqli_close($conn);  
            // Sucesso: Redireciona com flag de sucesso  
            header("Location: ../fornecedores.php?id=" . $equipamento_id . "&msg=assoc_removida");  
            exit;  
        } else {  
            mysqli_stmt_close($stmt);  
        }  
    }  
    mysqli_close($conn);  
    header("Location: ../fornecedores.php?id=" . $equipamento_id . "&msg=erro");  
    exit;  
}
```

Figura 3 - Código para desassociação de fornecedores a equipamentos.

2.6. Listagem de dados

Em termos gerais, o conjunto de ficheiros responsáveis pelas listagens partilha uma mesma infraestrutura técnica e um propósito focado na visualização de dados do inventário hospitalar. Todos os scripts são páginas PHP híbridas que combinam lógica de backend com a renderização de tabelas e interfaces visuais no frontend recorrendo ao framework Bootstrap 5 e a ícones da biblioteca Font Awesome. Do ponto de vista da infraestrutura de dados, todos estabelecem exatamente a mesma ligação à base de dados remota MySQL utilizando a extensão `mysqli_connect` com parâmetros de porta e credenciais idênticos. A nível de arquitetura de software, todos utilizam o mecanismo clássico de renderização dinâmica em ciclos `while` com `mysqli_fetch_assoc`, tratando cenários em que as tabelas estão vazias através da exibição de mensagens amigáveis e personalizadas com ícones textuais de alerta. Adicionalmente, todos os ficheiros manipulam mensagens flash guardadas em sessões do PHP para exibir alertas visuais de sucesso ou de erro no topo da página e concluem a sua execução fechando de forma limpa as ligações abertas à base de dados com a função `mysqli_close`.

Por outro lado, os scripts diferenciam-se marcadamente pelo tipo de dados que expõem, pela complexidade das suas consultas SQL e pela separação de conceitos entre dados operacionais e dados arquivados. Metade dos ficheiros foca-se na listagem principal de registos que estão ativos e disponíveis no dia a dia da instituição, enquanto a outra metade atua estritamente como um arquivo histórico ou "papeleira de reciclagem", isolando os registos desativados por motivos de auditoria através da verificação da coluna de estado. As tabelas de fornecedores são as mais simples a nível relacional, limitando-se a ler campos diretos de uma única tabela por ordem alfabética. Em contrapartida, as listagens de componentes, garantias e documentação aumentam drasticamente a complexidade ao utilizarem junções de tabelas como `INNER JOIN` e `LEFT JOIN` para cruzar dados operacionais complexos, permitindo, por exemplo, que um documento técnico ou um contrato de manutenção exiba o nome real e o número de série do equipamento médico ao qual está estritamente indexado. Há ainda variações nos botões de ação e operações disponíveis em cada tabela; enquanto as listagens principais oferecem opções para visualizar, editar ou desativar registos através de links dinâmicos parametrizados com IDs, as páginas de arquivo histórico trocam estas opções por um botão único focado na reativação e restauro do registo para o estado operacional. Por fim, o script de equipamentos sobressai ao integrar dados de geolocalização hospitalar interna e o ficheiro de documentação técnica inclui atalhos interativos em JavaScript para a cópia imediata de caminhos de ficheiros para a área de transferência do utilizador.

2.7. Validação para funções de Administrador

Em funções como a edição do texto do frontend através do backend ou o acesso à lista de equipamentos eliminados/arquivados e a reativação dos mesmos há um script php que estabelece uma barreira de segurança que restringe o acesso a áreas críticas do sistema, permitindo a sua visualização exclusiva a utilizadores com privilégios de administração. Esta lógica depende diretamente do sucesso do processo de login prévio, momento no qual o sistema consulta a base de dados e anexa à sessão do utilizador o marcador que define o seu nível de autorização. Ao tentar aceder à página protegida, o script avalia se esse marcador específico está ausente ou se o seu valor é diferente de verdadeiro. Caso se confirme que o utilizador não possui permissões administrativas, o código impede de imediato o carregamento do resto da página, grava uma mensagem de aviso no sistema de sessões para informar o utilizador sobre a rejeição e força o seu redirecionamento automático de volta para o painel principal.

```
if (!isset($_SESSION['is_admin']) || $_SESSION['is_admin'] !== true) {  
    $_SESSION['mensagem_erro'] = "Acesso negado. Esta área está reservada";  
    header("Location: dashboard.php");  
    exit;  
}
```

Figura 4 - Verificação de privilégios de administrador.

2.8. Eliminação e recuperação de dados

Excluindo o ficheiro eliminar_associacao_fornecedor.php, todos os restantes apresentam uma estrutura muito semelhante: iniciam sessão, verificam a autenticação do utilizador, ligam-se à base de dados, validam o identificador recebido e utilizam prepared statements para garantir maior segurança. Além disso, implementam uma estratégia de soft delete, alterando o estado dos registos para "Inativo" nos ficheiros de eliminação e para "Ativo" nos de reativação, sem remover dados da base de dados.

As principais diferenças estão relacionadas com as entidades geridas e as respetivas regras de negócio. Os ficheiros trabalham sobre tabelas distintas, como equipamentos, componentes, fornecedores, localizações, documentação e garantias, apresentando informações específicas de cada registo antes da confirmação da operação. Alguns casos incluem validações adicionais, como a verificação de associações existentes em fornecedores e localizações, impedindo a inativação quando existem dependências. Os ficheiros de garantias destacam-se ainda pela utilização de JOINS para apresentar informação complementar dos equipamentos associados.

Em termos funcionais, os ficheiros de reativação diferem apenas por procurarem registos inativos e alterarem o seu estado para ativo. Assim, todos partilham a mesma arquitetura e metodologia de funcionamento, variando apenas nas tabelas manipuladas, nos dados apresentados ao utilizador e nas validações específicas exigidas por cada entidade.

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['confirmar_arquivar'])) {  
  
    // ALTERADO: Troca de DELETE para UPDATE (Soft Delete)  
    $sql_update = "UPDATE componentes_associados SET estado = 'Inativo' WHERE id = ?";  
    $stmt_update = mysqli_prepare($conn, $sql_update);  
  
    if ($stmt_update) {  
        mysqli_stmt_bind_param($stmt_update, "i", $id);  
  
        if (mysqli_stmt_execute($stmt_update)) {  
            $_SESSION['mensagem_sucesso'] = "Componente desativado e arquivado com sucesso!";  
            mysqli_close($conn);  
            // Redireciona diretamente para o histórico de inativos conforme solicitado  
            header("Location: ../listar/lista_componentes_inativos.php");  
            exit();  
        } else {  
            $_SESSION['mensagem_erro'] = "Erro técnico ao tentar arquivar o componente na base";  
        }  
        mysqli_stmt_close($stmt_update);  
    } else {  
        $_SESSION['mensagem_erro'] = "Erro interno ao preparar o arquivamento.";  
    }  
  
    mysqli_close($conn);  
    header("Location: ../listar/lista_componentes.php");  
    exit();  
}
```

Figura 5 - Aplicação de soft delete a componentes.

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['confirmar_reativacao'])) {  
  
    $sql_update = "UPDATE componentes_associados SET estado = 'Ativo' WHERE id = ?";  
    $stmt_update = mysqli_prepare($conn, $sql_update);  
  
    if ($stmt_update) {  
        mysqli_stmt_bind_param($stmt_update, "i", $id);  
  
        if (mysqli_stmt_execute($stmt_update)) {  
            $_SESSION['mensagem_sucesso'] = "O componente '" . $componente['designacao_componente'] .  
            " foi reativado com sucesso!";  
            mysqli_close($conn);  
            // Redireciona de volta para a listagem principal operacional (Ativos)  
            header("Location: ../listar/lista_componentes.php");  
            exit();  
        } else {  
            $_SESSION['mensagem_erro'] = "Erro técnico ao tentar restaurar o componente.";  
        }  
        mysqli_stmt_close($stmt_update);  
    } else {  
        $_SESSION['mensagem_erro'] = "Erro interno ao processar a reativação.";  
    }  
  
    mysqli_close($conn);  
    header("Location: ../listar/lista_componentes_inativos.php");  
    exit();  
}
```

Figura 6 - Reativação de componentes eliminados.

2.9. Edição de Equipamentos, Componentes, Garantias, Localizações, Fornecedores e Documentos

Em termos de semelhanças, todos os ficheiros para edição partilham uma arquitetura híbrida idêntica, funcionando simultaneamente em dois modos operacionais dependendo do tipo de requisição HTTP recebida. Quando acedidos inicialmente através do método GET, todos extraem e convertem o parâmetro id do URL para garantir que é um número inteiro válido e executam uma consulta à base de dados para pré-preencher os campos do formulário com os valores atuais. Quando o utilizador submete as alterações, os scripts mudam para o modo POST, recolhendo, higienizando com trim e validando os novos dados antes de os gravar. A infraestrutura de dados é exatamente a mesma para todos, utilizando a mesma ligação remota MySQL e os mesmos parâmetros de porta e credenciais. Do ponto de vista visual e interativo, todos os formulários são construídos dinamicamente com o framework Bootstrap 5, recorrem à biblioteca Font Awesome para os ícones dos botões, utilizam a função htmlspecialchars na injeção de variáveis para mitigar ataques de Cross-Site Scripting (XSS), disponibilizam um botão "Cancelar" que reconduz o utilizador em segurança para a respetiva listagem principal e encerram explicitamente a ligação ao servidor através de mysqli_close no final da página.

Por outro lado, as diferenças concentram-se na complexidade das consultas SQL, na manipulação de tipos de dados específicos e na validação de campos obrigatórios no backend. Os ficheiros `editar_localizacao.php` e `editar_fornecedor.php` destacam-se pela simplicidade relacional, trabalhando apenas com campos de texto diretos das suas tabelas de origem, embora o script de fornecedores inclua uma estrutura condicional robusta para preencher o formulário usando o operador de coalescência nula (`??`) e dados guardados na superglobal `$_POST` caso ocorra uma falha de validação. O script `editar_componente.php` diferencia-se por incluir uma consulta que expõe e vincula o componente ao ID do equipamento-pai ao qual está acoplado. A complexidade aumenta nos ficheiros `editar_documentacao.php`, `editar Equipamento.php` e `editar_garantia.php`, que interagem com o tipo de dados `date` e manipulam lógicas de negócio avançadas. O script de documentação valida e exibe datas de emissão e de validade, aplicando filtros adicionais para garantir que as alterações ocorrem apenas em documentos com estado ativo. O script de equipamentos introduz ciclos dinâmicos `foreach` no HTML para pré-selecionar opções guardadas em campos do tipo `ENUM` (como a criticidade e o estado do aparelho) e gere a chave estrangeira da sua localização geográfica. Por fim, o ficheiro `editar_garantia.php` sobressai por implementar uma lógica condicional de visibilidade no formulário, validando se a periodicidade contratual (trimestral, semestral ou anual) e os dados da entidade responsável devem ser guardados ou anulados com base na ativação ou desativação do marcador que indica a existência de um contrato de manutenção técnica ativo.

2.10. Edição de texto do Frontend através do Backend

O funcionamento da edição do texto do frontend através do backend baseia-se numa arquitetura dinâmica e centralizada na base de dados, permitindo que as alterações feitas no painel de administração modifiquem o conteúdo visível para o utilizador final sem que seja necessário alterar manualmente o código estrutural da página principal.

O motor de todo este processo reside no ficheiro `exibir_texto.php`, que atua como uma ponte de leitura e biblioteca auxiliar; assim que é incluído, ele liga-se ao servidor MySQL remota, extrai todos os registos estruturados em pares de chaves e conteúdos da tabela `textos_interface` e armazena-os num array em memória, disponibilizando adicionalmente a função global `exibir_texto()` que serve para imprimir de forma segura o texto correspondente a uma chave ou recorrer a uma frase padrão caso esta ainda não exista na base de dados. Por sua vez, a página pública `index.php` representa o frontend do projeto e, em vez de codificar títulos ou parágrafos de forma fixa no HTML, inclui o script anterior logo no seu topo e invoca a função `exibir_texto()` em pontos estratégicos do código, como na secção de apresentação e na descrição do sistema, garantindo que o conteúdo renderizado no navegador é sempre a versão mais recente guardada na base de dados.

A alteração prática deste conteúdo ocorre através do ficheiro de backend `editar_texto_frontend.php`, que funciona como o painel de controlo exclusivo para administradores, exigindo uma validação de sessão rigorosa antes de expor o formulário de edição. Este script de administração utiliza igualmente o ficheiro de leitura para pré-preencher as caixas de texto e áreas de comentário com os dados atuais e, assim que o administrador submete o formulário via método `POST`, processa as novas strings recebidas, aplicando lógicas de atualização na base de dados para sobrescrever as chaves antigas, o que faz com que, na próxima vez que um visitante aceder ao `index.php`, o ciclo de leitura capture imediatamente os novos dados e atualize visualmente o frontend de forma instantânea.

```
if (!$conn) {
    die("Falha na ligação: " . mysqli_connect_error());
}

// Procura todos os textos da base de dados
$stmt = mysqli_query($conn, 'SELECT chave, conteudo FROM textos_interface');
$textos = [];

while ($row = mysqli_fetch_assoc($stmt)) {
    $textos[$row['chave']] = $row['conteudo'];
}

// Função auxiliar para evitar erros caso a chave não exista na BD
function exibir_texto($chave, $texto_padrao) {
    global $textos;
    return isset($textos[$chave]) ? htmlspecialchars($textos[$chave], ENT_QUOTES, 'UTF-8') : $texto_padrao;
}
?>
```

Figura 7 - Função `exibir_texto`.

2.11. Pesquisa avançada

A pesquisa avançada permite consultar o inventário hospitalar através de múltiplos filtros combinados, estando o acesso restrito a utilizadores autenticados. Após validar a sessão, o sistema estabelece ligação à base de dados e configura a codificação UTF-8 para garantir o correto tratamento de caracteres especiais.

Os critérios de pesquisa incluem código interno, designação, marca, modelo, número de série, serviço hospitalar, estado, fornecedor, categoria e criticidade. Antes do processamento, os dados são limpos e protegidos através de sanitização e prepared statements. Complementarmente, o JavaScript remove espaços desnecessários, quebras de linha, tabulações e espaços duplicados antes do envio do formulário. A consulta SQL é construída dinamicamente, adicionando apenas os filtros preenchidos pelo utilizador. As pesquisas textuais utilizam LIKE e utf8mb4_general_ci, permitindo pesquisas sem distinção entre maiúsculas, minúsculas e acentos. A consulta recorre ainda a JOINS para obter dados de localizações e fornecedores, utilizando GROUP_CONCAT para reunir vários fornecedores associados ao mesmo equipamento.

O sistema permite ordenar os resultados por designação, código interno, criticidade ou data de registo, em ordem crescente ou decrescente, recorrendo a uma lista de colunas previamente autorizadas para garantir segurança.

Após a execução da consulta, os resultados são apresentados numa tabela com informações essenciais dos equipamentos, incluindo código interno, designação, marca, modelo, número de série, serviço, criticidade e estado. Os níveis de criticidade e os estados são destacados visualmente através de badges coloridas. Caso não existam correspondências para os filtros aplicados, é apresentada uma mensagem informativa ao utilizador.

Em suma, a funcionalidade combina controlo de acesso, limpeza e validação de dados, consultas SQL seguras e dinâmicas, integração de informação de várias tabelas e apresentação organizada dos resultados, oferecendo uma pesquisa flexível, segura e eficiente do inventário hospitalar.

2.12. Vista detalhada de Equipamentos

A vista detalhada de equipamentos permite consultar toda a informação associada a um equipamento específico do inventário hospitalar. O sistema começa por validar a autenticação do utilizador e verificar se foi fornecido um identificador válido do equipamento. Em seguida, estabelece ligação à base de dados e executa várias consultas para obter os dados do equipamento, da sua localização, dos fornecedores associados, das garantias e contratos de manutenção, dos componentes e da documentação técnica.

A informação é apresentada de forma organizada, incluindo dados técnicos como designação, código interno, marca, modelo, número de série, fabricante, datas relevantes, criticidade, localização, estado atual e observações. São também mostrados os fornecedores associados, com os respetivos contactos e funções, bem como os detalhes da garantia e dos contratos de manutenção, indicando automaticamente se a garantia se encontra válida ou expirada.

A página inclui ainda uma secção para listar componentes e acessórios associados ao equipamento, bem como uma área dedicada à documentação técnica, onde são apresentados os documentos registados, as respetivas datas de validade e opções para abertura ou descarregamento dos ficheiros.

Em síntese, esta funcionalidade reúne numa única página toda a informação técnica, comercial, contratual e documental relacionada com um equipamento, proporcionando uma visão completa e centralizada do ativo hospitalar.

3. Base de Dados

3.1. Modelo relacional

O modelo relacional do sistema de base de dados é estruturado através de tabelas (relações) que se interligam por chaves primárias e chaves estrangeiras, organizando a informação em três núcleos principais: Gestão de Equipamentos e Componentes, Apoio Operacional (Localizações, Fornecedores e Documentos) e Controlo de Acesso/Interface.

No núcleo central, a tabela equipamentos atua como o eixo principal do sistema, possuindo uma relação de 1 para Muitos (1:N) com a tabela componentes_associados, o que significa que um equipamento pode ter vários sub-módulos ou peças acopladas, mas cada componente pertence exclusivamente a um equipamento-pai (equipamento_pai_id). De forma semelhante, estabelece-se uma relação de 1 para Muitos (1:N) com a tabela documentacao, permitindo indexar múltiplos ficheiros, como manuais de serviço e certificados de calibração (equipamento_id), a um único dispositivo médico. A acompanhar o ciclo de vida do aparelho, a tabela garantias_contratos liga-se a equipamentos numa relação de 1 para 1 (1:1) através do campo equipamento_id (que possui uma restrição de unicidade UNIQUE), garantindo que cada máquina do inventário dispõe apenas de um registo de cobertura de garantia ou contrato de manutenção técnica ativo.

No bloco de suporte logístico e geográfico, a tabela localizaciones conecta-se a equipamentos numa relação de 1 para Muitos (1:N) através da chave estrangeira localizacao_id; isto dita que uma localização ou sala específica do hospital pode albergar diversos equipamentos em simultâneo, mas cada equipamento está alocado a apenas um espaço físico de cada vez. No que concerne às entidades externas, a tabela fornecedores desdobra-se em duas direções relacionais distintas. Por um lado, liga-se a garantias_contratos numa relação de 1 para Muitos (1:N) através do campo entidade_responsavel_id, associando uma empresa externa como prestadora de assistência de vários contratos. Por outro lado, liga-se diretamente a equipamentos através da tabela de associação equipamento_fornecedor, que materializa uma relação de Muitos para Muitos (M:N); esta tabela de junção permite que um equipamento seja assistido ou fornecido por várias empresas e que cada fornecedor atenda a múltiplos dispositivos, discriminando o papel da entidade através do campo tipo_fornecedor integrado na própria chave primária composta.

Por fim, o módulo de segurança e customização do sistema opera de forma isolada do inventário técnico. A tabela utilizadores armazena as credenciais gerais de acesso e estende a sua identidade para a tabela administradores numa relação de herança/especialização de 1 para 1 (1:1), onde a chave estrangeira utilizador_id é simultaneamente a chave primária da tabela filha, garantindo que um utilizador comum pode ou não ser promovido a administrador. Totalmente autónoma e sem chaves estrangeiras vinculadas, a tabela textos_interface funciona como um dicionário global do tipo chave-valor, guardando de forma normalizada as strings textuais dinâmicas que preenchem o frontend da aplicação.

3.2. Restrições de Integridade

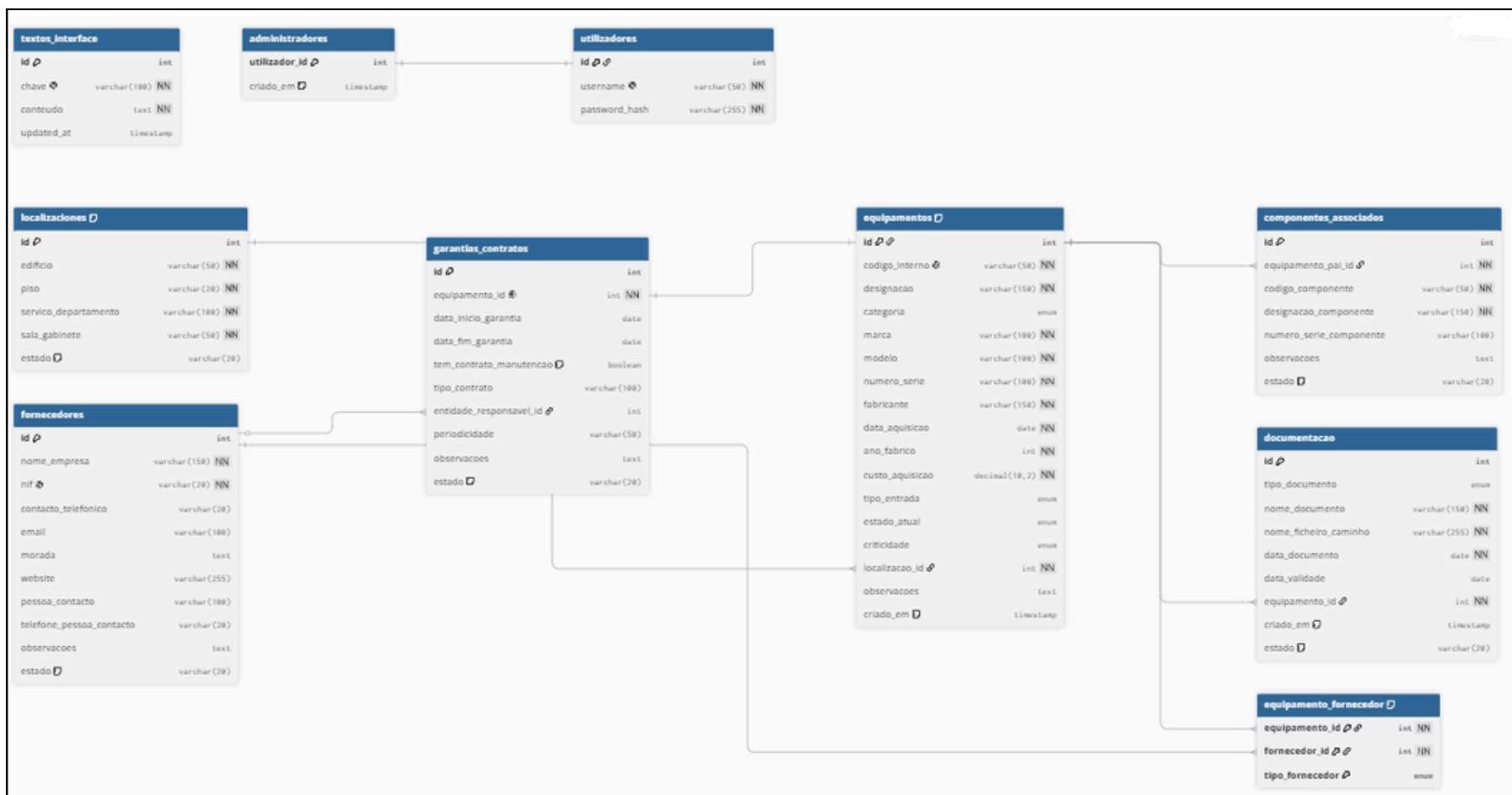


Figura 8 - Notação Crow's foot.

O esquema de base de dados apresentado estrutura um sistema de inventário hospitalar e gestão técnica baseado em quatro grandes categorias de restrições de integridade que asseguram a consistência e a proteção dos dados. A integridade de entidade é garantida através de chaves primárias numéricas com incremento automático na generalidade das tabelas, destacando-se a tabela de ligação entre equipamentos e fornecedores que recorre a uma chave primária composta por três campos para inviabilizar associações idênticas duplicadas. Ao nível do domínio e obrigatoriedade, o uso da instrução não nula salvaguarda o preenchimento de campos fundamentais para o negócio, enquanto a definição de valores padrão por omissão inicializa registos como ativos ou falsos e a aplicação de listas fechadas do tipo enumerado padroniza e limita as opções textuais permitidas em colunas críticas, como categorias e estados dos aparelhos. A unicidade dos dados é blindada por restrições simples que impedem a duplicação de registos de utilizadores, números de identificação fiscal e códigos de inventário, complementada por índices de unicidade compostos que bloqueiam a repetição física da mesma localização hospitalar ou a inserção de equipamentos com a mesma marca, modelo e número de série em simultâneo. Por fim, a integridade referencial governa os relacionamentos através de chaves estrangeiras com três comportamentos distintos em caso de eliminação: o apagamento em cascata encarrega-se de limpar automaticamente sub-módulos, garantias ou permissões se o registo-pai deixar de existir; a restrição de eliminação protege o sistema ao proibir a remoção de salas ou fornecedores que possuam vínculos operacionais ativos; e a anulação de vínculo limpa e redefine o campo do fornecedor para nulo nos contratos se a empresa for apagada, mantendo o histórico do contrato sob gestão interna do hospital.

3.3. Exemplos de Consultas SQL

```
$sql_garantia = "SELECT gc.*, f.nome_empresa AS entidade_responsavel_nome  
FROM garantias_contratos gc  
LEFT JOIN fornecedores f ON gc.entidade_responsavel_id = f.id  
WHERE gc.equipamento_id = ?";
```

Figura 9 - Consulta de junção de tabelas.

```
"SELECT id, codigo_componente, designacao_componente, numeroSerie_componente, observacoes  
FROM componentes_associados  
WHERE equipamento_pai_id = ?  
ORDER BY codigo_componente ASC";
```

Figura 10 - Consulta SELECT parametrizada com cláusulas WHERE e ORDER BY.

```
"INSERT INTO equipamentos (  
codigo_interno, designacao, categoria, marca, modelo,  
numero_serie, fabricante, data_aquisicao, ano_fabrico,  
custo_aquisicao, tipo_entrada, localizacao_id, estado_atual,  
criticidade, observacoes  
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
```

Figura 11 - Inserção de valores à tabela equipamentos

No desenvolvimento do projeto foram utilizadas várias consultas SQL para realizar operações de inserção e consulta de dados na base de dados. Um dos exemplos corresponde a uma consulta INSERT, utilizada para registar novos equipamentos na tabela equipamentos. Esta consulta permite armazenar informações como código interno, designação, categoria, marca, modelo, número de série, fabricante, datas de aquisição e fabrico, custo, tipo de entrada, localização, estado atual, criticidade e observações. A utilização de parâmetros (?) indica que a inserção é executada através de prepared statements, aumentando a segurança e prevenindo ataques de injeção SQL.

Foram também utilizadas consultas SELECT para recuperar informação específica da base de dados. Um exemplo é a consulta à tabela componentes_associados, que obtém os componentes ligados a um determinado equipamento através do campo equipamento_pai_id, apresentando os resultados ordenados pelo código do componente. Esta consulta é utilizada para visualizar a composição e os acessórios associados a cada equipamento.

Outro exemplo corresponde à consulta das garantias e contratos, que utiliza um LEFT JOIN entre as tabelas garantias_contratos e fornecedores. Esta junção permite apresentar simultaneamente os dados da garantia e o nome da entidade responsável associada ao contrato, mesmo quando não existe um fornecedor registado para esse efeito. Desta forma, é possível reunir informação proveniente de diferentes tabelas numa única consulta.

Estas consultas demonstram a utilização dos principais mecanismos SQL do projeto, incluindo inserção de dados, filtragem através de cláusulas WHERE, ordenação com ORDER BY, junções entre tabelas através de JOINS e execução segura através de prepared statements.

4. Conclusão

O desenvolvimento do Sistema de Inventário Biomédico e de Dispositivos de Apoio à Saúde (SIBDAS) permitiu criar uma solução completa para a gestão centralizada de equipamentos hospitalares, respetivos componentes, fornecedores, contratos de manutenção, garantias e documentação técnica. O sistema foi concebido com o objetivo de melhorar a organização da informação, facilitar a consulta de dados e apoiar a gestão eficiente do parque tecnológico hospitalar.

Ao longo do projeto foram implementadas diversas funcionalidades que abrangem todo o ciclo de vida dos equipamentos, desde a sua inserção e associação a fornecedores até à edição, pesquisa, consulta detalhada, desativação e reativação dos registos. Foram igualmente desenvolvidos mecanismos de autenticação e controlo de acessos, garantindo que apenas utilizadores autorizados podem aceder às áreas restritas do sistema e executar operações críticas.

A nível técnico, foi dada especial atenção à segurança da aplicação através da utilização de sessões, validação de permissões, prepared statements para prevenção de ataques de SQL Injection, validação e sanitização de dados introduzidos pelos utilizadores e aplicação de medidas adicionais de proteção contra acessos indevidos. A estrutura da base de dados foi projetada segundo o modelo relacional, recorrendo a chaves primárias, chaves estrangeiras e restrições de integridade que asseguram a consistência e fiabilidade da informação armazenada.

Destaca-se ainda a implementação de funcionalidades avançadas, como a pesquisa dinâmica com múltiplos filtros, a visualização detalhada de equipamentos, a gestão documental e a edição de conteúdos do frontend através do backend, proporcionando uma experiência de utilização mais completa e flexível.

Em suma, os objetivos definidos para o projeto foram alcançados com sucesso, resultando numa aplicação web robusta, segura e adaptada às necessidades de gestão de inventário hospitalar. O sistema desenvolvido constitui uma ferramenta capaz de apoiar eficazmente o controlo dos ativos tecnológicos de uma instituição de saúde, contribuindo para uma melhor organização, rastreabilidade e disponibilidade da informação.